

Smart Shopping & Price-Comparison AI Agent

A practical system for product research, price comparison, and purchase guidance

SEARCH

Find suitable products

COMPARE

Evaluate price and value

RECOMMEND

Support the final decision

Project Type	Autonomous multi-agent shopping assistant
Framework	CrewAI
Language	Python
Main Output	Streamlit interface with evidence-based Markdown and PDF reports
Student Name	Reaksmey Rin
Course	CS 342 AI
Instructor	Rathanak Sreang

Prepared as a technical project report

PROJECT SUMMARY

Executive Summary

Online shopping offers more choice than ever, but it also creates a practical problem: buyers must move between websites, compare incomplete information, judge seller reliability, and decide whether a low price is actually a good deal. The Smart Shopping & Price-Comparison AI Agent was designed to handle that research process in a structured way.

The system is built with CrewAI and uses three specialized agents. The first agent searches for products that match the user's request and verifies selected pages. The second agent compares the available prices, specifications, sellers, warranties, delivery details, and overall value. The final agent selects the best-supported option. The tasks exchange validated Pydantic objects, and a deterministic Python callback renders the final result as matching Markdown and PDF reports with alternatives and source URLs. A Streamlit web interface now lets a local user submit a request, follow the run status, read the Markdown report, and download either report format from a browser.

Project idea in one sentence

A user describes what they want to buy, and the system researches the market, compares realistic options, and explains which product offers the strongest value for that user.

Report at a Glance

Section	Focus
1–3	Background, problem, and objectives
4–6	Technology stack, agent design, and workflow
7–10	Input, reliability, efficiency, and output
11–16	Web interface, evaluation, safe buying, limitations, improvements, and conclusion

BACKGROUND

1. Introduction

Finding a suitable product online is rarely a single-step task. A buyer may begin with a budget, a preferred brand, or a specific use case, but still needs to compare many details before deciding. The same product can appear under different names, at different prices, and with different warranty or shipping conditions.

This project addresses that gap by turning the shopping process into a small research workflow. Instead of generating an immediate answer from memory, the system searches current web sources, organizes the findings, and produces a recommendation that can be checked by the user. The workflow can be started from the command line or through a local Streamlit interface designed for a clearer, beginner-friendly experience.

2. Problem Statement

Consumers often spend a significant amount of time comparing products across online stores. Important details are frequently scattered between product pages, marketplace listings, seller policies, and delivery

information. A low listed price may also be misleading when shipping, warranty, or seller reliability are considered.

Core problem

The buyer does not only need the cheapest listing. The buyer needs the option that best balances price, quality, suitability, seller reliability, and purchasing conditions.

3. Project Objectives

The project was designed around one main goal: make product research easier without hiding uncertainty.

- Understand the user's product type, budget, intended use, preferred features, and location.
- Find several realistic products from accessible online sources.
- Compare price, specifications, availability, seller information, warranty, and delivery details.
- Identify the normal price range and the strongest value option.
- Recommend alternatives instead of forcing a single choice.
- Provide direct source URLs, label missing evidence clearly, and make the resulting reports easy to read and download in a browser.

SYSTEM DESIGN

4. Technology Stack

Technology	Role in the Project	Why It Was Chosen
CrewAI	Coordinates agents, tasks, context, and execution order.	Provides a clear structure for multi-step work.
DeepSeek V4 Flash	Performs reasoning, comparison, and recommendation.	Suitable for language-based analysis within the crew.
ExaSearchTool	Searches the web and returns product-related sources.	Provides current information and source URLs.
ScrapeWebsiteTool	Reads selected product pages for verification.	Helps confirm details beyond a search-result snippet.
Pydantic	Defines and validates the structured data exchanged between tasks.	Provides consistent fields, types, and predictable task handoffs.
Python + uv	Runs the project and manages dependencies.	Keeps the environment reproducible and easy to maintain.
PyMuPDF	Renders the validated recommendation as a printable PDF.	Creates a shareable report without another model call.
Streamlit	Provides the local browser interface, progress feedback, report preview, and downloads.	Integrates directly with the existing Python workflow and is easy to run for a project demonstration.

API keys are stored in a local `.env` file and are not hardcoded into the source code. This keeps credentials separate from the project logic and reduces the risk of accidentally publishing secrets.

5. Agent Roles

Agent	Main Responsibility	Search Access
Product Search Specialist	Finds suitable products and verifies selected pages.	Exa search and page scraping
Price Comparison Analyst	Compares price, value, seller conditions, and missing information.	One targeted verification search when necessary
Product Recommendation Advisor	Selects the best-supported option and explains alternatives.	No new search; uses previous task results

6. Sequential Workflow

01	The user submits {product_query} through Streamlit or the CLI.
02	The search agent returns a validated ProductSearchResult.
03	The comparison agent returns a validated PriceComparisonResult.
04	The advisor returns a validated FinalRecommendationResult.
05	A Python callback renders the structured result to outputs/report.md and outputs/report.pdf.
06	Streamlit previews the Markdown result and provides Markdown/PDF downloads.

6.1 Architecture Overview

The diagram below shows how the browser interface starts the existing CrewAI workflow without changing its responsibilities. Search tools are available only where new evidence is needed, while each completed stage passes a typed result to the next agent. The validated final result is rendered once and returned to Streamlit for preview and download.

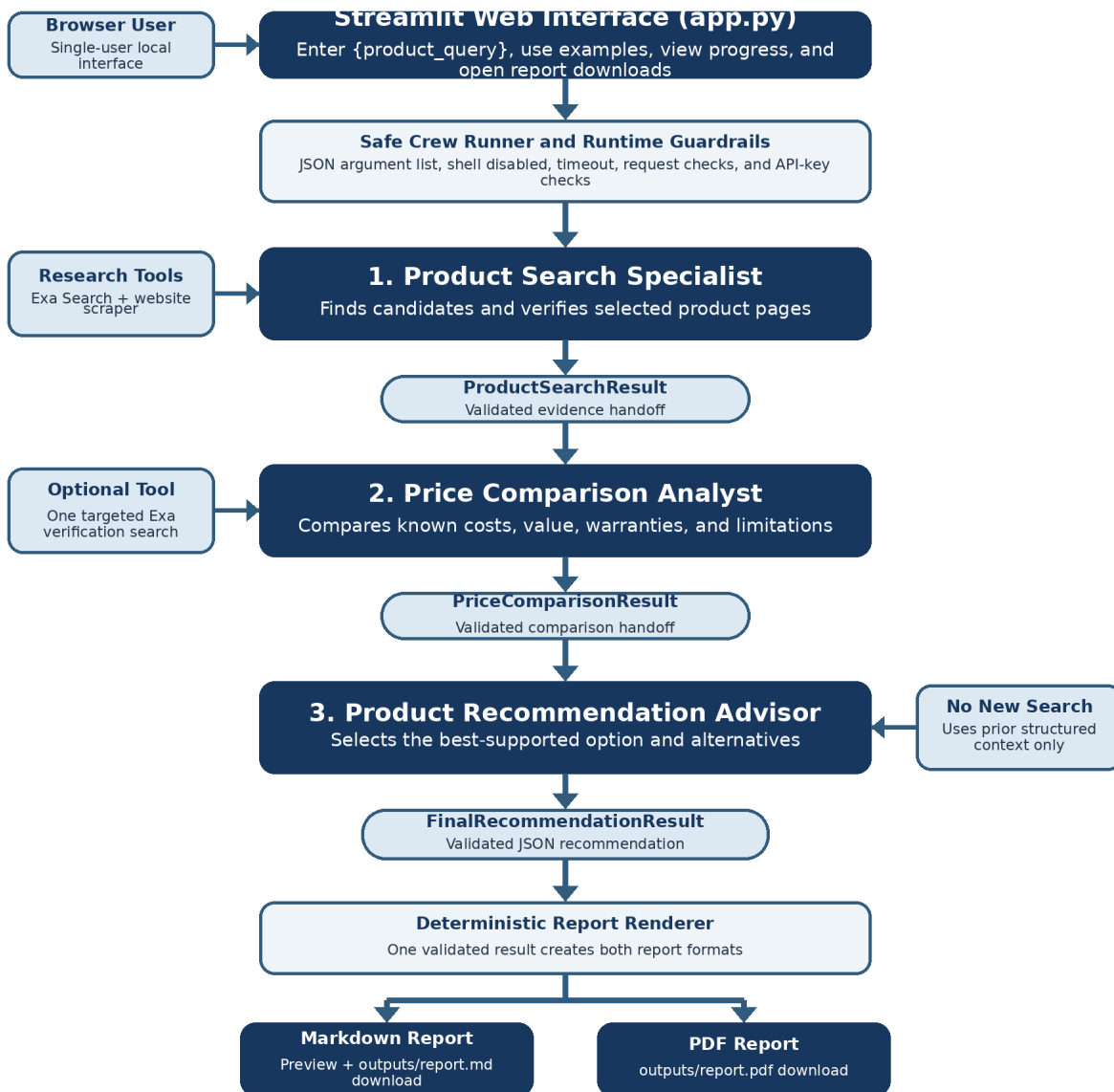


Figure 1. System architecture and structured data flow.

6.2 Key Design Decisions

- Specialized roles: each agent has one clear responsibility, making the workflow easier to understand, test, and explain.
- Sequential execution: comparison depends on research evidence, and recommendation depends on both earlier results.
- Structured handoffs: Pydantic contracts reduce missing fields and prevent free-form reports from becoming the next agent's only input.
- Restricted tool access: the advisor cannot start a conflicting search, and the analyst uses search only for targeted verification.
- Deterministic reporting: Python creates Markdown and PDF from one validated result, avoiding an additional model call.
- Thin interface wrapper: Streamlit invokes the same crew command, so the browser experience reuses the existing schemas, safety checks, and reporting callback instead of duplicating them.

6.3 Structured Agent Handoffs

The contracts make the educational purpose of each stage visible. They validate field names and data types while keeping uncertain facts explicit.

Stage	Structured Contract	Information Passed Forward
Research	ProductSearchResult	Requirements, product evidence, source URLs, research issues, and shortfall
Comparison	PriceComparisonResult	Per-product costs, value analysis, price ranges, best-value deal, and limitations
Recommendation	FinalRecommendationResult	Selected option, alternatives, confidence, checklist, and numbered sources

Simplified handoff example

ProductSearchResult(products=[...]) → PriceComparisonResult(comparisons=[...]) →

FinalRecommendationResult(recommendation=..., confidence=...)

OPERATION

7. User Input

The main input is passed through the `{product\_query}` placeholder. A short request can work, but the recommendation becomes more useful when the user includes a budget, intended use, important features, and location.

Example input

Find the best gaming laptop under \$1,000 for university work and gaming. It should have at least 16 GB of RAM and be available to a buyer in Cambodia.

8. Reliability Rules

Shopping information changes quickly, so the project treats verification as part of the workflow. The agents are instructed not to present unsupported details as confirmed facts, and every task must return data that matches its assigned Pydantic schema.

- Every recommendation should include direct source URLs.
- Prices, sellers, availability, warranty, delivery, and purchase links must come from accessible evidence.
- When a fact cannot be confirmed, the report uses the label “Could not verify.”
- The final advisor should not perform a second independent search that conflicts with earlier task context.
- The buyer is reminded to recheck price and stock before payment.
- Structured outputs reject unexpected fields, require HTTP(S) product URLs, use exact decimal values for money, and limit ProductSearchResult to five product records.
- The three task contracts are ProductSearchResult, PriceComparisonResult, and FinalRecommendationResult.
- Structural validation improves consistency but does not independently prove that every shopping claim is true; the buyer must still recheck time-sensitive facts.

9. Efficiency and Cost Control

Control	Configured instruction or limit
Focused product searches	At most two Exa searches (task instruction)
Verified product pages	At most five pages plus one replacement (task instruction)
Additional analyst search	At most one targeted search (task instruction)
Recommendation research	No search tools; uses prior structured results
Retries and iterations	Bounded in agent settings

10. Final Output

The final advisor produces a validated `FinalRecommendationResult`. A deterministic Python callback then renders the same structured data into `outputs/report.md` and `outputs/report.pdf`. Both formats contain the same recommendation and sources, and PDF generation does not require another agent or model call. Streamlit previews the Markdown result and offers both files as downloads.

- Summary of the user's requirements
- Recommended product and reason
- Verified price, seller, and product URL
- Top alternatives
- Advantages, limitations, and recommendation confidence
- Unverified information
- Safe-buying checklist
- Numbered source list

11. Streamlit Web Interface

The project now includes `app.py`, a Streamlit front end that makes the multi-agent workflow usable from a local browser. It is deliberately a thin interface: the app collects the shopping request and invokes the same `crewai run` workflow used by the command line. The agent prompts, Pydantic schemas, guardrails, and report callback remain unchanged.

11.1 User Experience and Application Flow

Interface Element	User Experience	Implementation Behavior
Request entry	Text area plus three example prompts	Stores the selected or typed request in Streamlit session state
Run control	Compare Products button and several-minute progress spinner	Disables controls during a run to reduce accidental duplicate requests
Crew invocation	No command-line knowledge required	Serializes <code>product_query</code> as JSON and passes an argument list with shell disabled
Results	Readable recommendation appears in the browser	Renders <code>outputs/report.md</code> and offers Markdown/PDF download buttons
Failures	Clear messages for rejected requests, missing keys, timeouts, and other errors	Shows a short redacted diagnostic view and saves local troubleshooting output

11.2 Safety, Scope, and Local Operation

The interface builds the subprocess command as a list, uses JSON serialization for the request, sets `shell=False`, captures output, and applies a 600-second timeout. The existing `before-kickoff` callback still rejects prohibited or underspecified requests and checks required API keys before paid work. The first release is intentionally local and single-user because all runs share the same report filenames.

```
Launch the web interface
uv sync --locked
uv run streamlit run app.py
```

The browser normally opens at `http://localhost:8501`.

EVALUATION

12. Educational Evaluation

The offline test suite was run on 25 July 2026 using `python -m unittest discover -s tests -v`. All 52 tests passed without using Exa Search or DeepSeek credits. The tests demonstrate that the project is configured correctly, handles its structured outputs consistently, and exercises the interface’s testable helper functions.

Evaluation Area	Evidence Checked	Result
Crew configuration	Three agents, tools, models, task order, and context references	Passed
Runtime guardrails	Missing keys, prohibited requests, vague requests, and valid scoped requests	Passed

Evaluation Area	Evidence Checked	Result
Structured data	Exact decimals, HTTP(S) URLs, extra-field rejection, and five-product limit	Passed
Crew loading	Schemas and final report callback resolve from the JSONC configuration	Passed
Report generation	Readable Markdown and PDF, source links, and invalid JSON rejection	Passed
Streamlit helpers	Input validation, safe argument construction, secret redaction, timeout handling, and failure classification	Passed
Interface smoke check	Initial page, four buttons, one request field, empty-input error, and example selection	Passed offline
Live factual accuracy	Prices, stock, seller claims, and delivery information can change online	Manual verification required

Evaluation interpretation

Passing tests shows that the software structure, validation, and report generation behave as expected in offline checks. The smoke check confirms that the Streamlit page loads and its non-paid interactions work. It does not replace a live end-to-end shopping run or prove that every online claim is permanently correct, so source links and the final buying checklist remain important.

13. Safe-Buying Guidance

The project supports the decision, but it does not complete a purchase automatically. Before paying, the user should complete the following checks:

1. Open the original product page and confirm the current price.
2. Check whether the exact model and configuration are still in stock.
3. Review warranty, return, and refund conditions.
4. Confirm the final shipping cost and delivery estimate.
5. Check seller reputation and avoid untrusted payment channels.
6. Use a secure payment method and keep the order confirmation.

14. Current Limitations

The system can reduce research time, but it cannot remove every uncertainty. Some websites block automated access, some sellers publish incomplete specifications, and shipping costs may only appear during checkout. Prices and stock can also change after the report is generated. Pydantic validates the structure of task outputs, not the truth of every claim. The current Streamlit version is intended for one local user: all runs share outputs/report.md and outputs/report.pdf, so concurrent browser sessions could overwrite each other's files.

How the project should be understood

This is a decision-support system, not an automatic purchasing system. It organizes evidence and explains trade-offs, while the final decision and payment remain with the user.

15. Future Improvements

Improvement	Purpose
Price history	Track price changes and identify whether a current discount is genuine.
Cambodian store coverage	Add more local sellers and delivery information.
Currency handling	Convert prices and show the exchange-rate date.
Multi-user web deployment	Add background jobs, per-run report storage, authentication, and deployment controls for concurrent users.
Saved preferences	Remember preferred brands, budgets, and previous searches.
Notifications	Alert users when a product reaches a chosen price.

16. Conclusion

The Smart Shopping & Price-Comparison AI Agent demonstrates how a multi-agent workflow can support a practical consumer task. The project separates product discovery, price analysis, and recommendation into three clear roles, which makes the process easier to understand and easier to improve.

With one user request, the system can research products, compare available evidence, identify value, and produce a purchasing recommendation with sources. Its most important design choice is how it handles uncertainty: the agents exchange structured results, unexpected data shapes are rejected, and missing details are marked instead of guessed. The user still performs the final verification before payment.

APPENDIX

Project Configuration Summary

Item	Configuration
Project name	Smart Shopping & Price-Comparison AI Agent
Execution model	Sequential multi-agent workflow
Agents	Product Search Specialist; Price Comparison Analyst; Product Recommendation Advisor
Primary input	{product_query}
Search tool	ExaSearchTool
Verification tool	ScrapeWebsiteTool
Language model	DeepSeek V4 Flash
Output files	outputs/report.md; outputs/report.pdf
Credential storage	.env
Structured outputs	ProductSearchResult; PriceComparisonResult; FinalRecommendationResult
Report renderer	tools.reporting.save_recommendation_report
Web interface	Streamlit 1.60.0 in app.py

Sample Request

Find the best smartphone under \$500 available in Cambodia. Compare at least five options, show verified prices and sources, and explain which model offers the best overall value.

Suggested Run Command

```
cd ~/smart_shopping_price_comparison_agent
uv sync --locked
uv run streamlit run app.py

CLI alternative:
uv run crewai run
```